基于 Clang/LLVM的语言处理流程与 SysY手写实现

编译原理第一次作业实验报告 作者：LIZZYGREAT 日期：2026 年 10 月 7 日

摘要

本文以一个含函数、while、if/else 、取模和整数输入输出的 SysY风格程序为贯穿案例，使用 Clang 14 观察预处理、LLVM IR生成、x86-64 汇编、ELF目标文件和显式链接过程。随后根据源程序语义独立手写 LLVM IR 与 RV64汇编，并分别链接运行库验证。六组输入在宿主机 reference 、手写 LLVM和 QEMU/RISC-V 上均得到 1、5、8、16、21、85。优化对比显示，Clang -O2消除了该模块中的 alloca/load/store ，并在 x86-64 上生成四 lane 向量运算；RV64目标保留标量循环并内联 calc。最后依据 LLVM与 AscendNPU-IR官方材料分析 VecAdd的多层 MLIR lowering，明确该部分属于资料分析，没有在本机运行 CANN编译器或 Ascend设备。

关键词： 编译器；Clang；LLVM IR；RISC-V；ABI；MLIR；lowering

1 引言

编译器把源程序逐步转换为可执行程序。理解每一步的输入、输出和符号关系，有助于分清预处理、编译、汇编和链接的职责。LLVM IR适合观察语言结构如何变为控制流和算术指令；RISC-V 汇编则展示调用约定、寄存器和栈帧如何落实到目标架构。

本实验固定一个程序，避免不同算法导致比较基准变化。程序令 i 从 1 递增到输入 n，若 i 为偶数则将 2i 加入 sum，否则将 i 加入 sum，最后输出 sum。程序覆盖局部整型变量、赋值、加法、乘法、取模、比较、条件分支、循环、函数参数、函数返回和运行库调用。所有实现都使用同一输入集合验证。

2 实验对象与环境

程序语义可写为：

```
sum = 0
for i = 1..n:
if i % 2 == 0: sum += 2  i
else:          sum += i
return sum
```

工作环境为 Ubuntu 22.04.5 on WSL2，Clang 14.0.0，GCC 11.4，QEMU 7.0.0 RISC-V unknown-elf GCC 15.1，Flex 2.6.4、Bison 3.8.2。课程目录已有编译器和 SysY runtime，本实验复用其 libsysy_riscv.a ；没有重新安装或改动该目录。Clang的 RISC-V参考目标和手写目标统一使用 rv64gc/lp64d 。

3 Clang语言处理流程

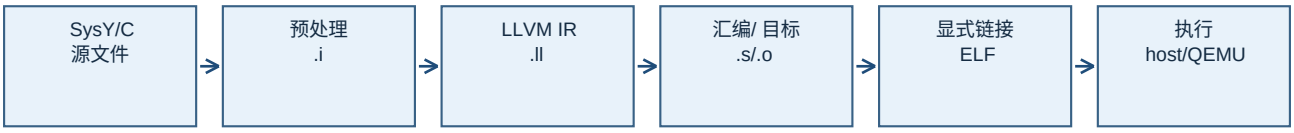


图 1：从源程序到运行文件的实验流程

3.1 预处理

命令 clang -E source/example_clang.c -o generated/example.i
生成预处理文件。本例没有宏、条件编译或头文件，因此 .i 主要保留原 C
语句和源文件行标记。预处理器解释预处理指令并输出供后续前端处理的源文本；本例中的 if 、while
语义由后续编译阶段处理。

3.2 编译与 LLVM IR

命令 clang -S -emit-llvm -O0 source/example_clang.c -o generated/example.ll 输出 LLVM IR。Clang
前端完成词法、语法和语义处理，再生成 IR；LLVM中端可在 IR 上优化，后端按目标架构选择指令。本次 -O0
IR 中，局部变量由 alloca/load/store 表示，比较使用 icmp，while和 if/else 通过 br
与基本块表达，函数调用和返回体现为 call/ret 。

3.3 汇编与目标文件

clang -S -O0 生成宿主机 x86-64 汇编 generated/example_x86.s。Clang -c 将 C文件继续编译、汇编成 ELF64
可重定位目标文件 generated/example.o；该文件的 ELF类型为 REL，包含
.text 、符号表和重定位项。符号表显示 calc、main已定义，getint 和 putint
仍为未定义符号。它们的调用位置需要由链接器结合运行库解析。

3.4 链接

本实验分别生成 example.o与 host_runtime.o，再用 Clang驱动显式链接。日志显示 Clang调用
/usr/bin/ld ，并加入 CRT启动对象和 -lc 。host_runtime.o 定义 getint/putint ，系统 C库解析
scanf/printf 。链接器应用重定位、合并输入节区并生成最终可执行 ELF。单独的 example.o
既没有完整的外部符号定义，也不是可直接执行的进程映像。

阶段	命令或产物	本实验观察
预处理	clang -E；example.i	保留 C语句并带行标记；本例无宏/ 头文件展开
LLVM IR	clang -S -emit-llvm -O0；example.ll	alloca、load/store 、icmp、br、call、ret
汇编	clang -S -O0；example_x86.s	x86-64 指令和调用序列
汇编器	clang -c ；example.o	ELF REL 节区、符号和重定位
链接器	example.o + host_runtime.o	解析 runtime/C 库符号并生成可执行 ELF

4 LLVM IR控制流与手写实现

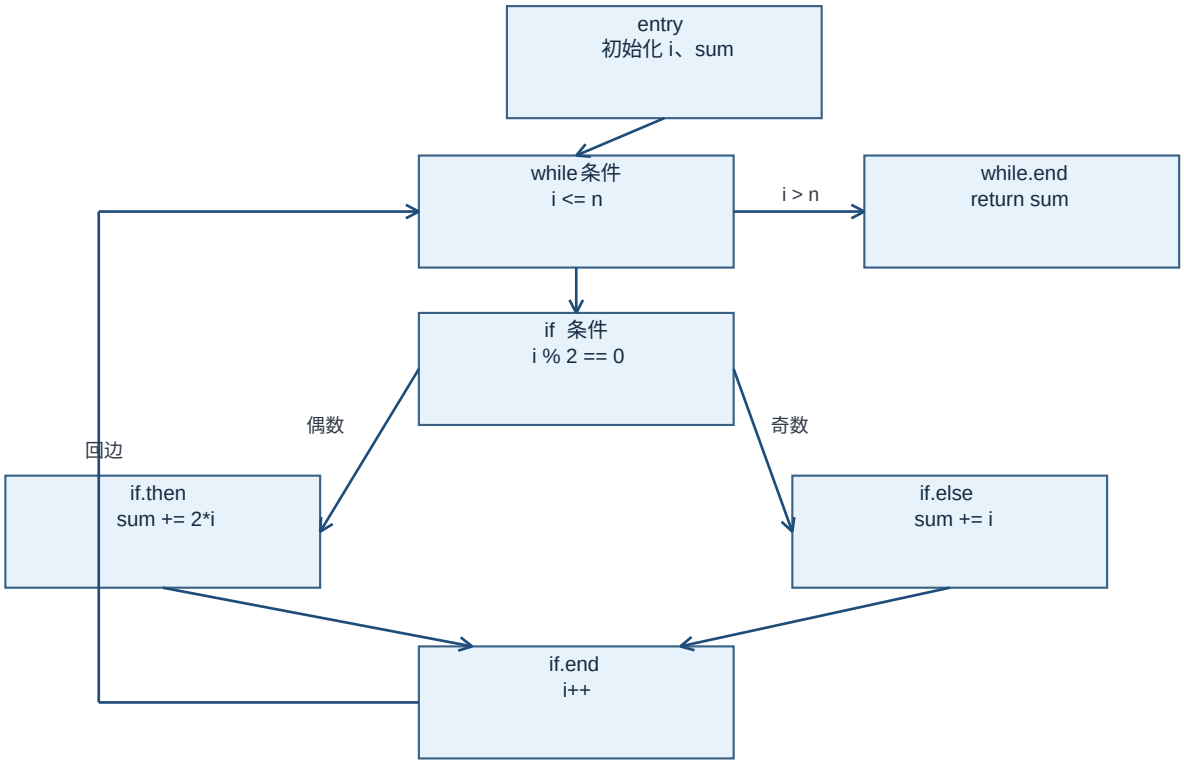


图 2：calc 的基本块与回边

Clang -O0生成的 calc 使用数字标签。入口块初始化 i 和 sum后进入 %5；%5 用 icmp sle计算 i <= n。真分支进入 %9 判断 i%2，两个分支 %13、%18 分别计算偶数项和奇数项，并汇入 %22。%22 执行 i++ 后无条件跳回 %5，构成 while 回边；假分支到 %25 读取 sum并返回。

源程序结构	LLVM指令	作用
局部变量声明/ 赋值	alloca、store	为 n/i/sum 分配栈槽并写初值或新值
读取变量	load	取 i、n、sum的当前值
加法、乘法、取模	add、mul、srem	计算计数器、累加项与奇偶性
比较	icmp sle、icmp eq	判断循环范围和余数是否为零
if/while	br、基本块	根据条件转移，合并分支并形成循环回边
函数关系	define、call、ret	定义 calc/main，调用 getint/calc/putint 并返回

手写 llvm/manual.ll根据源程序和 CFG独立组织 entry 、while.cond、if.cond 、if.then 、if.else 、if.end 、while.end七个命名块，使用 alloca/load/store 保存局部变量。Clang接受该 IR 并将其与 host_runtime.c 链接；六组输入全部通过。

5 RISC-V汇编与 ABI

RISC-V参考汇编由 Clang 14交叉生成。手写 riscv/manual.s 将 n、i、sum保存于 calc 栈帧；while条件用 blt n,i,while_end判断退出；remw与 beqz 选择奇偶分支；偶数项用 slliw 乘 2，累加用 addw，循环尾递增 i 并跳回条件块。

寄存器/ 规则	本实验用法
a0	第一个整型参数与整型返回值；getint 、calc、putint 之间传值
t0-t2	calc 内的 caller-saved 临时值

寄存器/ 规则	本实验用法
s0	calc 的 frame pointer ；按 ABI 保存并恢复
sp	栈指针；calc/main 分别保留 32/16 字节且保持 16 字节对齐
ra	返回地址；main跨越函数调用保存/ 恢复

手写汇编通过 riscv64-unknown-elf-gcc 汇编和链接已有 SysY runtime，采用课程配置的 -static -mcmodel=medany -Ttext=0x90000000 生成 RISC-V ELF executable，再由 QEMU用户态模拟器运行。课程 runtime 的析构函数会把 TOTAL计时摘要写入 stderr ；程序数值输出仍与 reference 相同。

6 运行验证

输入 n	Reference	手写 LLVM	手写 RISC-V/QEMU	结果
1	1	1	1	PASS
2	5	5	5	PASS
3	8	8	8	PASS
4	16	16	16	PASS
5	21	21	21	PASS
10	85	85	85	PASS

最终结果由 outputs/final-validation.md 和 logs/09_final_validation.log 留证。比较脚本检查每个实现是否等于独立固定的期望值，不只比较三者之间是否相同。

7 优化观察：-O0 与 -O2

LLVM IR指令数量	-O0	-O2
alloca	6	0
load	11	0
store	9	0

-O2 将可变局部量改写为 SSA值和循环 phi，消除了显式栈槽读写，并把余数/if 奇偶计算改成位运算与移位。x86-64 IR 出现四 lane <4 x i32>运算和 vector reduce ；汇编中可见 XMM与 paddd，calc 仍有外部可见定义，但 main内的调用已被内联。x86 汇编行数从 87 变为 385，增加来自向量化循环体和内联代码，行数本身不表示速度或代码尺寸。

RV64目标为 rv64gc，不包含 RVV扩展。其 -O2 汇编仍为标量循环，没有向量寄存器指令；优化将 calc 内联进 main，并化简奇偶累加。RV64汇编行数从 91 变为 56。本实验没有测量执行时间，因此不据此推断性能提升。

8 MLIR / AscendNPU VecAdd

MLIR 以 operation/value/block/region 和 dialect 描述不同抽象层。VecAdd的高层语义是 C[j]=A[j]+B[j] ，元素类型和 tensor shape 可保留在 Arith/Tensor 或 Linalg 表示中。HFusion 对算子图执行融合、tiling 与调度；HIVM示例则显式表示 GM输入、UB暂存、GM到 UB的 load、hivm.hir.vadd 和写回 GM，HACC属性标记 device kernel 入口，配套 host wrapper 完成 ACL初始化、数据传输、launch 和同步。[1-7]

层	VecAdd表达	关注点
Arith/Tensor	张量形状与逐元素 add	运算语义、元素类型和 shape
Linalg/HFusion	张量算子及融合/tiling	算子组合和执行计划

层	VecAdd表达	关注点
HIVM	GM/UB、load、vadd、store	昇腾存储层次与硬件操作
HACC	entry 与 HOST/DEVICE属性	主机/设备边界和 kernel 启动
LLVM IR/后端	逐步转换为低层指令/目标代码	控制流、地址、调用约定和机器码

此阶段是官方文档与源码分析，没有安装 CANN，也没有运行 bishengir-opt 、bishengir-compile 或 Ascend NPU。公开 VecAdd样例展示 HIVM kernel .o 和 CANN host wrapper；本文没有把未观察到的 LLVM IR dump 写成实测中间产物。LLVM dialect 到 LLVM IR的渐进转换机制依据 LLVM官方文档说明。[3]

9 结论

通过同一 SysY程序，实验观察了 Clang从预处理文本、LLVM IR、宿主机汇编到 ELF 目标文件和链接结果的流程。手写 LLVM IR和 RV64 汇编均遵循源程序控制流与函数语义，并通过六组输入验证。优化对照显示，优化会改变值表示、循环形态和目标指令选择，且不同后端得到不同的向量化结果。MLIR 的多层 dialect 保留张量/算子信息，并在 lowering 中逐步引入内存与硬件细节；Ascend示例分析以公开资料为依据，未进行本机 NPU编译或运行。

参考文献

1. LLVM Project. MLIR Language Reference. https://mlir.llvm.org/docs/LangRef/ （访问日期：2026-10-07） 2. LLVM Project. MLIR Rationale. https://mlir.llvm.org/docs/Rationale/Rationale/ （访问日期：2026-10-07） 3. LLVM Project. Targeting LLVM IR from MLIR. https://mlir.llvm.org/docs/TargetLLVMIR/ （访问日期：2026-10-07） 4. Ascend. AscendNPU-IR Framework Integration. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/developer_guide/conversion/framework_interface.md （访问日期：2026-10-07） 5. Ascend. AscendNPU-IR Interface API. https://github.com/Ascend/AscendNPU-IR/blob/master/docs/source/en/developer_guide/conversion/interface_api.md（访问日期：2026-10-07） 6. Ascend. HIVM VecAdd add.mlir. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/add.mlir （访问日期：2026-10-07） 7. Ascend. HIVM VecAdd README and host wrapper. https://github.com/Ascend/AscendNPU-IR/blob/master/bishengir/test/Integration/HIVM/VecAdd/README.md（访问日期：2026-10-07） 8. Compiler-Labs. docs/requirements.md and docs/执行流程.md, local course assignment and experiment plan, accessed 2026-10-07.